Technical Specification: Enterprise Retrieval-Augmented Generation (RAG) Reference Architecture

1. Executive Summary

Retrieval-Augmented Generation (RAG) combines external knowledge retrieval with large language model (LLM) generation. This pattern mitigates hallucinations, reduces retraining costs, and grounds AI responses in verifiable, domain-specific enterprise data.

2. Core Architecture Pipeline

The end-to-end RAG workflow consists of five sequential stages:

- **Data Ingestion:** Raw documents (PDFs, HTML, Markdown) are parsed to extract structural text, tables, and metadata.
- **Text Chunking:** Content is split into smaller, context-preserving segments (recommended size: 500–800 tokens with a 10–15% overlap).
- **Vector Embedding:** Chunks are mapped to high-dimensional numerical vectors using embedding models (e.g., OpenAI text-embedding-3 or open-source alternatives like MiniLM).
- **Vector Storage & Indexing:** Vectors are stored in optimized vector databases (e.g., FAISS, Chroma, Qdrant, or Pinecone) for high-speed similarity searches.
- **Retrieval & Generation:** User queries are vectorized to retrieve top-k relevant context chunks via cosine similarity, which are then injected into an LLM prompt template to generate grounded answers.

3. Key Configuration Parameters

- **Chunk Size:** 512 tokens
- **Chunk Overlap:** 64 tokens
- **Similarity Metric:** Cosine Similarity
- **Top-K Retrieval:** 4 chunks
- **Embedding Dimension:** 1536-d

4. Evaluation and Guardrails

To ensure production reliability, systems must measure:

- **Context Precision:** Are the retrieved chunks relevant to the user query?

- **Faithfulness:** Does the generated answer derive strictly from the retrieved context without introducing external assumptions?
- **Answer Relevance:** Does the final output directly address the user's intent?

If you need this text formatted as a **Python script to load and chunk this exact text**, or converted into a **JSON format for a vector database**, let me know how you'd like to proceed!